axiateam
RDT company

Módulo 8

Clases, Interfaces y Herencia

Curso: TypeScript

120 min

Objetivos de Aprendizaje

- Definir e implementar **clases** con tipos completos
- Crear **interfaces** para definir contratos
- Implementar **herencia** de clases
- Utilizar **modificadores de acceso** (public, private, protected)
- Aplicar **abstracción** con clases e interfaces

1. Contexto

Programación Orientada a Objetos

Por qué OOP con TypeScript

TypeScript trae POO a JavaScript con tipos estáticos:

```
class User {
  private id: string;
  public name: string;
  protected email: string;

  constructor(id: string, name: string, email: string) {
    this.id = id;
    this.name = name;
    this.email = email;
  }

  public greet(): string {
    return `Hola, soy ${this.name}`;
  }
}
```

2. Clases Básicas

Clases con Tipos

Las clases combinan datos (propiedades) y comportamiento (métodos) en una sola unidad con tipos.

```
class Task {
  // Propiedades tipadas
  id: string;
  title: string;
  private _status: string = "pending";

  // Constructor
  constructor(id: string, title: string) {
    this.id = id;
    this.title = title;
  }

  // Getter
  get status(): string {
    return this._status;
  }

  // Método
  complete(): void {
```

Propiedades `readonly` y `static`

`readonly` impede modificaciones; `static` permite acceso sin instanciar.

```
class Config {
  readonly version: string = "1.0";
  static instance: Config | null = null;

  private constructor() {}

  static getInstance(): Config {
    if (!Config.instance) {
      Config.instance = new Config();
    }
    return Config.instance;
  }
}

const config = Config.getInstance();
console.log(config.version);
```

Parameter Properties

El constructor puede declarar propiedades directamente.

```
class User {
  constructor(
    public name: string,
    private id: string
  ) {}
}
```

Getters y Setters

Permiten acceso controlado a propiedades.

```
class Task {
  private _status: string = "pending";

  get status(): string {
    return this._status;
  }

  set status(value: string) {
    if (!["pending", "inProgress", "completed"].includes(value)) {
      throw new Error(`Estado inválido`);
    }
    this._status = value;
  }
}
```

Lab: Clase TaskManager

Objetivo: Implementar una clase manager para el sistema de tareas.

1. Crea la clase `TaskManager`
2. Añade métodos de filtrado
3. Añade estadísticas

Código del Lab

```
class TaskManager {
  private tasks: Task[] = [];

  addTask(title: string, priority: TaskPriority): Task {
    const task = createTask(title, priority);
    this.tasks.push(task);
    return task;
  }

  getTasks(): Task[] {
    return [...this.tasks];
  }

  findTaskById(id: string): Task | undefined {
    return this.tasks.find(t => t.id === id);
  }

  removeTask(id: string): boolean {
    const index = this.tasks.findIndex(t => t.id === id);
```

3. Interfaces

Definiendo Contratos

Una interface define qué propiedades y métodos debe tener una clase, sin implementación.

```
// Interface para repositorio
interface ITaskRepository {
  findAll(): Task[];
  findById(id: string): Task | null;
  save(task: Task): Task;
  delete(id: string): boolean;
}

// Implementación
class InMemoryTaskRepository implements ITaskRepository {
  private tasks: Task[] = [];

  findAll(): Task[] {
    return [...this.tasks];
  }

  findById(id: string): Task | null {
    return this.tasks.find(t => t.id === id) || null;
  }
```

Propiedades `readonly` y `?`

`readonly` marca una propiedad como inmutable; `?` la hace opcional.

```
interface User {
  readonly id: string;
  name: string;
  email?: string;
}

// Error: no se puede asignar a propiedad readonly
// user.id = "2"; // Error!
```

Herencia de Interfaces (`extends`)

Una interface puede extender a otra, heredando sus propiedades.

```
interface Animal {
  name: string;
}

interface Pet extends Animal {
  ownerId: string;
}

interface Dog extends Pet {
  breed: string;
}
```

Index Signatures

Permite acceder a propiedades con claves dinámicas.

```
interface StringMap {
  [key: string]: string;
}

const strings: StringMap = {
  key1: "value1",
  key2: "value2"
};
```

Lab: Interfaz de Servicio

Objetivo: Crear interfaz de servicio para el sistema de tareas.

1. Define la interfaz `ITaskService`
2. Implementa la clase `TaskServiceImpl`
3. Usa la interfaz en lugar de la implementación

Código del Lab

```
interface ITaskService {
  create(title: string, priority: TaskPriority): Task;
  getAll(): Task[];
  getById(id: string): Task | null;
  update(task: Task): Task | null;
  delete(id: string): boolean;
  filterByStatus(status: TaskStatus): Task[];
  filterByPriority(priority: TaskPriority): Task[];
}

class TaskService implements ITaskService {
  constructor(private repository: ITaskRepository) {}

  create(title: string, priority: TaskPriority): Task {
    const task = createTask(title, priority);
    return this.repository.save(task);
  }

  getAll(): Task[] {
```

4. Herencia

Herencia de Clases

`extends` permite crear una clase hija que hereda propiedades y métodos de una clase padre.

```
// Clase base
class Person {
  constructor(public name: string) {}
  greet(): string {
    return `Hola, soy ${this.name}`;
  }
}

// Clase derivada
class Employee extends Person {
  constructor(name: string, public employeeId: string) {
    super(name);
  }

  greet(): string {
    return `${super.greet()} y soy empleado #${this.employeeId}`;
  }
}
```

Method Overriding y `super`

`super` permite invocar el método de la clase padre; `override` reescribe el comportamiento.

```
class Base {
  greet(): string {
    return "Hola";
  }
}

class Derived extends Base {
  greet(): string {
    return super.greet() + ", mundo!";
  }
}

const d = new Derived();
d.greet(); // "Hola, mundo!"
```

Lab: Tipos Especiales de Tareas

Objetivo: Implementar herencia para tipos especializados de tareas.

1. Crea clase base abstracta `BaseTask`
2. Crea clase `RegularTask` que hereda de BaseTask
3. Crea clase `UrgentTask` que hereda de BaseTask
4. Crea clase `RecurringTask` que hereda de BaseTask

Código del Lab

```
abstract class BaseTask {
  constructor(
    public id: string,
    public title: string,
    public status: TaskStatus = "pending"
  ) {}

  abstract getPriority(): number;

  complete(): void {
    this.status = "completed";
  }
}

class RegularTask extends BaseTask {
  constructor(
    id: string,
    title: string,
    public priority: TaskPriority
```

Resumen

Puntos Clave

- **public**: accesible desde cualquier lugar
- **private**: solo accesible dentro de la clase
- **protected**: accesible desde la clase y sus subclases
- **interfaces** definen contratos sin implementación
- **extends** para herencia de clases
- **implements** para implementar interfaces

Ejercicio Integrador

Sistema de Biblioteca

Caso 2: Biblioteca

Objetivo: Aplicar OOP al sistema de biblioteca.

1. Crea clase base `User` con herencia para `Member`
2. Implementa interfaces para repositorios
3. Crea clase `LibraryManager`

Trabajo en grupos de 2-4 personas.

Tips y Gotchas

✅ Tips

- `interface` para contratos que pueden implementarse
- `type` para uniones, interserciones y alias
- `implements` es para interfaces, `extends` para clases

⚠️ Gotchas

- `abstract` classes no se pueden instanciar directamente
- Multiple interfaces: `class A implements B, C`
- `protected` permite acceso en subclases, `private` no

Referencias

Recursos Adicionales

- Classes
- Interfaces

Gracias!

Módulo 8 Completado

Siguiente: Módulo 9 - Genéricos